Take-Home Assignment: Build a Usage-Metered SaaS with Flexprice

Role: Software Engineer **Time allowed:** 4–5 days from receiving this assignment **Tech stack:** Your choice (Flexprice provides JavaScript, Python, and Go SDKs, plus a plain REST API) **Final step:** A live demo on your machine (localhost), walking us through the product and the code

---

1. Overview

We want to see how you approach building a small but real product and integrating it with a third-party billing platform. You will build a **minimal SaaS application** and integrate it with **Flexprice** — an open-source platform for usage metering, feature management, and billing — which you will **self-host locally**.

By the end, your SaaS should have **plans with feature-based pricing**, **usage-based metering with limits**, and a small **pricing experiment** comparing two pricing models.

This is not a UI/design test. A clean, functional interface is enough. We care about:

- Correct, well-structured integration with an external API
- Sensible data modeling (your users ↔ Flexprice customers/subscriptions)
- How you handle limits, edge cases, and failures
- Your ability to reason about pricing and usage data
- Your ability to explain your decisions in the demo

---

2. Part 0 — Self-host Flexprice

Set up Flexprice locally by following the official guide: 👉 https://docs.flexprice.io/docs/getting-started/self-hosting-guide

In short:

```
git clone https://github.com/flexprice/flexprice
cd flexprice
```

```
make dev-setup
```

You will need **Docker + Docker Compose**, **Go 1.23+**, and **make** (Linux, macOS, or WSL). Once running, the Flexprice API is available at `http://localhost:8080` and the default local API key is `sk_local_flexprice_test_key` (sent via the `x-api-key` header).

Checkpoint: you can hit `GET http://localhost:8080/v1/customers` with the API key and get a valid response.

> If you hit setup issues, document what went wrong and how you fixed (or worked around) it — troubleshooting is part of the assignment.

---

3. Part 1 — Build a minimal SaaS

Build a small web application with a metered core action. Pick one of these (or propose your own of similar scope):

- **AI-ish text tools** — e.g. summarize / rewrite / word-stats on submitted text (a fake/stubbed "model" is fine; we are not testing ML)
- **Link shortener** — create short links, track redirects
- **Image utility** — resize/compress uploaded images
- **Notes/API product** — a REST API product where API calls are the metered unit

Minimum requirements:

1. **Sign up / log in** — simple auth is fine (session or JWT; no OAuth needed).
2. On signup, the user is **registered as a customer in Flexprice** (use `external_customer_id` to link your user to the Flexprice customer) and **subscribed to the Free plan** automatically.
3. At least **one core action** a user performs repeatedly (summarize a text, shorten a link, resize an image, make an API call). This is your metered unit.
4. At least **one premium feature** that only paid-plan users can access (e.g. bulk mode, custom slugs, higher file size, export).

---

4. Part 2 — Feature-based pricing (plans & entitlements)

Model your pricing in Flexprice, not in hard-coded `if plan == "pro"` checks scattered through your app.

Requirements:

1. Create at least **two plans** in Flexprice: **Free** and **Pro** (a third plan is a bonus).
2. Define **features** in Flexprice and attach them to plans via **entitlements**:
   - At least one **boolean feature** (on/off — e.g. "custom slugs", "bulk mode") that gates your premium feature.
   - At least one **metered feature** (e.g. `link.shortened`, `text.summarized`) with **different usage limits per plan** (e.g. Free = 20/month, Pro = 1,000/month).
3. Your app must **check entitlements via the Flexprice API** to decide:
   - whether a user can access the premium feature, and
   - whether the user still has usage quota remaining.
4. Add a way to **upgrade a user from Free to Pro** in your app (a simple "Upgrade" button that switches the subscription is fine — **no real payment gateway needed**).
5. When a Free user hits a gated feature or an exhausted quota, show a **clear, friendly upgrade prompt** — not a 500 error.

Relevant docs:

- Features: https://docs.flexprice.io/docs/product-catalogue/features/create
- Linking features to plans: https://docs.flexprice.io/docs/product-catalogue/features/linking-to-plans
- Plans: https://docs.flexprice.io/docs/product-catalogue/plans/create
- Subscriptions: https://docs.flexprice.io/docs/subscriptions/customers-create-subscription

---

5. Part 3 — Usage metering

Requirements:

1. Every time a user performs the core action, **send a usage event to Flexprice** (event ingestion API/SDK) with the correct `event_name`, `external_customer_id`, and any properties your meter needs.
2. Configure the **meter/metered feature** in Flexprice with a sensible **aggregation** (COUNT is fine; SUM over a property like `credits` or `size_mb` is a bonus) and a **periodic reset** (per billing cycle).

3. Build a small **usage page/section in your app** showing the current user:
    - usage so far in the current period (fetched from Flexprice, not your own DB),
    - their limit, and how close they are to it,
    - their current plan and its features.
4. **Enforce the limit:** once a user exhausts their quota, block the action and show the upgrade prompt.

Relevant docs:

- Event ingestion: https://docs.flexprice.io/docs/event-ingestion/overview
- Creating a metered feature: https://docs.flexprice.io/docs/event-ingestion/creating-a-metered-feature
- Usage by subscription: https://docs.flexprice.io/api-reference/subscriptions/get-usage-by-subscription

Things we will look at closely:

- What happens if Flexprice is down or slow when a user performs an action? (Do you fail open or closed? Why?)
- Are events sent reliably (e.g. not silently dropped on errors)?
- Is there any double-counting or race condition around the limit check?

---

6. Part 4 — Pricing experiment

We want to see that you can use Flexprice for **pricing experiments**, not just billing plumbing.

Requirements:

1. In addition to your main pricing, model **one alternative pricing scheme** for the Pro plan's usage charge using a different Flexprice pricing model — e.g.:
    - **Flat per-unit** (0.01*peraction*)*vs* * *package* * (5 per 1,000 actions), or
    - flat per-unit vs **volume/tiered** pricing (first 10k at one rate, beyond that cheaper).
2. Write a small **simulation script** (any language) that:
    - creates 3–5 fake customers with different usage profiles (light, medium, heavy),
    - pushes a realistic burst of usage events for each into Flexprice,
    - pulls usage back from Flexprice and computes what each customer would pay under **pricing A vs pricing B**.

3. In your README, write **5–10 lines of analysis**: which pricing scheme earns more, which customer types win/lose under each, and which you would recommend and why.

Exact invoice generation in Flexprice is a bonus; computing the comparison from Flexprice usage data with your own script is perfectly acceptable.

---

7. Deliverables

1. **A Git repository** (GitHub/GitLab link, or a zip) containing:
    - Your SaaS application code
    - The pricing-experiment simulation script
    - Any setup/seed scripts you wrote for configuring Flexprice (plans, features, meters) — scripted setup via API is preferred over "I clicked in the dashboard", but a documented manual setup is acceptable
2. **A README** with:
    - What you built and why you chose it
    - Architecture: how your app, your DB, and Flexprice fit together (a simple diagram is welcome)
    - Exact steps to run everything from scratch on localhost
    - Your plan/feature/pricing design in Flexprice
    - The pricing experiment analysis (Part 4)
    - Known limitations and what you'd do with more time
3. **A live demo (~30 minutes)** on your machine, screen-shared:
    - Sign up a new user → show them appear as a customer in Flexprice
    - Perform metered actions → show usage climbing
    - Hit the Free limit → show the block + upgrade prompt
    - Upgrade to Pro → show the premium feature unlock and higher limit
    - Walk through the pricing experiment results
    - Walk through the code paths you're most and least proud of

---

8. Rules & notes

- **AI assistants are allowed** (Copilot, Claude, ChatGPT, etc.) — but you must fully understand every line you ship. The demo includes questions about your code and decisions, and "the AI wrote that" is not an acceptable answer.

- Keep it on **localhost** — no cloud deployment needed.
- No real payment processing — plan switching can be a plain API call.
- If you cannot finish everything, prioritize **Parts 0–3** and be upfront in the README about what's missing. A smaller, working, well-explained submission beats a large broken one.
- Questions during the assignment are welcome — asking good questions is a signal, not a weakness.

Good luck — we're looking forward to your demo!